

(https://
profile.intra.42.fr)

SCALE FOR PROJECT FT_ALITY (/PROJECTS/42CURSUS-FT_ALITY)

You should evaluate 2 students in this team



Git repository

git@vogsphere.42paris.fr:vogsphere/intra-uuid-9924c471-29f4-4f66-af20



Introduction

To ensure this evaluation goes smoothly, please respect the following set of rules :

- Please remain courteous, polite, respectful and constructive at all times during this exchange. The trust bond between the school's community and yourself depends on it.
- Should you notice any malfunctions within the submitted project, make sure you take the time to discuss those with the student (or group of students) being graded.
- Keep in mind that some subjects can be interpreted differently. If you come accross a situation where the student you're grading has interpreted the subject differently than you, try and judge fairly whether their interpretation is acceptable or not, and grade them accordingly. Our peer-evaluation system can only work if you both take it seriously.

Guidelines

- You may only evaluate whatever is in the GiT submission directory of the student you are grading.
- Make sure to check wether the GiT submission directory belongs to the student (or group) you're grading, and that it's the right project.
- Make sure no mischievous aliases have been used to trick you into correcting something that is not actually in the official submitted directory.
- Any script created to make this evaluation session easier - whether it was produced by you or the student being graded - must be checked rigorously in order to avoid bad surprises.
- If the student who is grading this project hasn't done the project him/herself yet, he/she must read the whole topic before starting

the evaluation session.

- Use the flags available to you on this scale in order to report a submission directory that is empty, non-functional, that contains a norm errors or a case of cheating, etc... In this case, the evaluation session ends and the final grade is 0 (or -42, in case of cheating). However, unless the student has cheated, we advise you to go through the project together in order for the two (or more) of you to identify the problems that may have led for this project to fail, and avoid repeating those mistakes for future projects.

Attachments

 subject.pdf (<https://cdn.intra.42.fr/pdf/pdf/149158/en.subject.pdf>)

Getting started

This section is dedicated to setup the evaluation and to test the prerequisites. It does not reward points, but if something is wrong at this step or at any point of the evaluation, the grade is 0, and an appropriate flag might be checked if needed.

Compliance to the rules

- The graded student (or team) work is present in his or her repository.
- The graded student (or team) should be able to explain their work at any point during the evaluation.
- The project is functional. If it is not, the evaluation ends here.
If the project uses a language that is not fully functional, ensure that:
- The general rules, as well as any day-specific rules, are always followed during the evaluation.
- The compiler does not report any errors or warnings.
- The Makefile does not relink.

 Yes

 No

Mandatory part

This section covers the most basic and fundamental features required for the subject. It must be completed perfectly to unlock bonuses.

Automaton

An automaton must be implemented somewhere in the code.
It could be a class, a module, or even hidden in a single, large file containing the entire project.
If you encounter the latter, please express disapproval.

The automaton must at least loosely correspond to the formal

definition provided in the subject. In other words, you should be

able to identify some states, some recognition states, and some

transitions.

Obviously, the student must have implemented the automaton

themselves. If they haven't (i.e., a magical automaton library

was imported, or the student cannot explain the code), that's a -42 and a light tap on the head.

☒ Yes

☐ No

Basic program behaviour

Now it's time to run the program! Quickly review the grammar files and try to get the program to recognize some moves.

For this step, let's keep things simple. Run the program with any grammar of your choice. It should display the keyboard mapping and wait for the user to press some keys. Check that the program correctly detects when keys are pressed and displays the user's input. Every now and then, you should also see the names of moves appear.

☒ Yes

☐ No

Small grammars

Create a small grammar file with just three or four moves, and run the program using it. You should be able to execute every move defined in the file. This step is also an opportunity to verify whether the keyboard mapping is correctly generated from the grammar file. It must be. If it isn't, this step will be considered a failure.

☒ Yes

☐ No

Homonymous rules

Create another small grammar file with several rules, but ensure

that all the rules are triggered by the same key combinations.

Run the program and execute the key combinations. You must be

able to see every rule name defined in the grammar.

☒ Yes

☐ No

Common prefixes

Create another small grammar file with several rules, ensuring

that all the rules share the same prefix. For example, you could

have something like:

Move A; a

Move B; a, b

Move C; a, b, a

The program should be able to display move names as they appear,

and the user must be able to execute every move in your grammar.

☒ Yes

☐ No

Error handling

No one enjoys nitpicking or spending hours hunting for bugs, only to end up giving the project a grade of 0.

However, functional programming is also about rigor, so we must at least test some simple erratic behaviors. In this section, as well as in the rest of the defense, any uncaught exception, infinite loop, freeze, or erratic behavior of any kind will result in the defense ending with a "Crash" status. This means the grade will be 0.

Command line arguments

Try running the program with the following scenarios: no

arguments, more than one argument, a file that does not exist,

and a file the program is not allowed to read (e.g.,

/etc/master.passwd). The program must handle each case and exit

gracefully.

☒ Yes

☐ No

Input

Test the program by pressing random keys on your keyboard.

Do this gently, but make sure to press keys outside the keyboard

mapping and attempt to execute inputs that do not correspond to

any specific move in the grammar. The program must reset the

input buffer when necessary and continue running regardless of

the input.

☒ Yes

☐ No

Bonus

Bonus section! This part rewards projects that have made special efforts to make the final program outstanding.

Bonus points

You can award bonus points as you see fit, but here are some

ideas for which you can grant points:

- Support for gamepads
- Graphical interface
- Debug mode
- Clean and organized code

Rate it from 0 (failed) through 5 (excellent)



Ratings

Don't forget to check the flag corresponding to the defense

✓ Ok

★ Outstanding project

Empty work

Incomplete work

Invalid compilation

Norme

Cheat

Crash

Incomplete group

Forbidden function

Can't support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

- API General
Terms of Use
(https://
profile.intra.42.fr/
legal/terms/33)

Declaration on
the use of
cookies (https://
profile.intra.42.fr/
legal/terms/2)

Privacy policy
(https://
profile.intra.42.fr/
legal/terms/35)

General term of
use of the site
(https://
profile.intra.42.fr/
legal/terms/6)

Internal Rules
(https://
profile.intra.42.fr/
legal/terms/7)

Terms of use for
video
surveillance
(https://
profile.intra.42.fr/
legal/terms/1)

Legal notices
(https://
profile.intra.42.fr/
legal/terms/3)